

NAN 2026 GAME × AI 해커톤 · 사전 과제

AI 활용 기술 문서

The Mastermind — 두뇌게임 아레나

플레이 링크 <https://hj0304.github.io/The-Mastermind/>

소스 저장소 <https://github.com/hj0304/The-Mastermind> (공개 · 커밋 기록 유지)

AI 도구 · 프롬프트 · 활용 내역 / 게임 AI 설계 / 외부 에셋·오픈소스 출처

1. 요약 — AI를 두 층위로 활용

본 프로젝트는 AI를 두 층위로 사용했습니다.

- 개발 도구로서의 AI** — Anthropic **Claude**(Claude Code 환경)를 기획·리서치·설계·구현·검증·배포 전 과정의 페어 프로그래머로 사용했습니다. 저장소의 커밋 본문에 `Co-Authored-By: Claude`로 기여가 표기되어 있습니다.
- 제품 속 AI** — 15종 게임 각각에 맞춤 설계된 대전 AI. 머신러닝 API 호출이 아니라 **게임별 알고리즘**(베이지안 추론, 알파베타 탐색, 행렬게임 혼합전략, 후보 소거 귀납 추론 등)을 순수 TypeScript로 구현했습니다. 여기에 더해 **은닉 정보 게임 6종에는 자가대국 강화학습(CFR 계열)**으로 훈련한 균형 근사 전략을 탑재했습니다 — 기존 최강 휴리스틱 AI를 상대로 **승률 62.9~80.8%**를 기록한 전략들입니다.

이 문서의 핵심 주장 — "AI를 썼다"가 아니라 **게임의 수학적 구조에 맞는 기법을 각각 선택하고, 그 선택이 옳았음을 수치로 검증**했다는 점입니다. 자가학습이 유효한 6종에는 CFR을, 완전정보 게임에는 탐색을, 추론 퍼즐에는 후보 소거를 적용한 근거를 3장에 정리했습니다.

2. AI 도구 사용 내역 (개발 과정)

항목	내용
도구	Claude Code (Anthropic Claude 모델) — CLI/데스크톱 에이전트 환경
활용 범위	원작 룰 리서치·검증, 게임 룰 명세 작성, 게임 엔진·AI·UI 구현, 온라인 전송 계층 설계, 버그 재현·수정, 브라우저 자동 검증, UI/UX·테마 시스템, 디자인 시안 분석·이식, 문서 작성
개발 방식	사람이 방향·요구사항·검수를 담당하고, AI가 구현·검증을 수행하는 페어 프로그래밍. GitHub Flow(기능 브랜치 → PR → main 병합)로 모든 변경이 PR 단위로 기록됨
기여 표기	AI가 작성에 참여한 커밋은 본문에 Co-Authored-By로 명시 (커밋 기록에서 확인 가능)

2.1 개발 워크플로우

- 룰 리서치** — 원작(더 지니어스 시리즈 방영 게임)의 룰을 나무위키 원문에서 확인하고, 구현 기준 명세(`docs/GAME_RULES.md`)로 정리. 구현 후에는 원작 문서 13건을 전수 재대조해 불일치를 수정.
- 구현** — 게임마다 엔진(순수 함수)·AI·화면을 분리 구현. 엔진은 부수효과 없는 순수 TypeScript라 AI 탐색과 온라인 동기화에 그대로 재사용.
- 검증** — AI가 브라우저를 직접 조작해 검증: 실제 릴레이를 경유하는 두 탭 온라인 대전, 360px 모바일 뷰포트 점검, 테마별 스크린샷 확인 등. 빌드는 타입 검사(`tsc`)를 포함.
- 배포** — main 병합 시 GitHub Actions가 GitHub Pages로 자동 배포.

2.2 AI 대상 주요 프롬프트/지시 사항 (실사용 예)

룰 검증 지시

"(나무위키 링크 첨부) 너가 게임 룰을 숙지하고 UI/UX를 수정했는지 의문이 들어. 위 링크를 살살이 뒤져서 모든 게임 룰을 숙지하고 다시 한번 볼래?" — 원작 문서 13건 전수 대조로 이어져, 원작의 '제시 즉시 게이미션' 공개 정보가 온라인 모드에서 누락된 결함을 발견·수정.

버그 리포트

"의심 윗놀이를 플레이했는데 윗판의 정가운데 지점에서 윗이 나왔는데 지름길을 타지 않는 케이스를 발견했어. 지름길을 탈 수 있도록 수정해줘." — 엔진 시뮬레이션으로 로직은 정상임을 확인, 완주 분기가 판 위에 표시되지 않던 UI 문제를 특정해 수정.

설계 요구

"디자인 시안 4종을 각 디자인의 느낌에 맞게 모든 페이지에 적용하고, 토글 버튼으로 유저가 테마를 선택할 수 있게 해줘. 지금 디자인도 저장해서 선택지에 포함해줘." — CSS 변수 기반 5종 테마 시스템으로 구현(게임 의미 색은 불변 원칙).

난이도 요구 → 자가학습 도입

"나는 이 게임의 AI를 정말 고난도로 만들고 싶었어. 알파고처럼 학습을 통해 강한 AI를 만들고 싶었는데 이거 가능해?" — 게임별 구조를 검토해 **은닉 정보 게임 5종에 CFR 계열이 적합**하다고 판단, 블라인드 포커부터 순차 도입해 5종을 탑재했고, 이후 신규 게임(블라인드 홀덤)에 같은 파이프라인을 재적용해 6종이 됐습니다(3.3절).

수치에 대한 검증 요구 (사람의 감독)

"62.9%면 학습 왜 했냐? 거의 반반보다 조금 앞선 거 아님?" — 승률의 절대값만으로는 크기를 판단할 수 없다는 지적. 이에 **기준선(기존 hard AI vs normal AI = 65.3%)을 실측**해 62.9%가 난이도 한 등급에 해당하는 격차임을 입증하고, 그 과정에서 균형 전략이 약한 상대를 덜 착취한다는 이론적 성질도 실측으로 확인.

콘텐츠 확충 요구

"히든 포물러 수식 좀 더 많이 만들었으면 좋겠는데? 한 200가지는 있어야 할 듯" — 손으로 나열하면 중복이 필연이므로 **계열 × 매개변수 생성 + 자동 감사 도구**로 접근. 감사가 실제 중복 8쌍을 검출(3.4절).

디자인 결과 반려 (사람의 감독)

"A안이 이거 말한 거 맞지? 왜 이따구로 뽑아놨어?" — 첨부한 디자인 시안 파일을 AI가 **텍스트만 추출**해 읽고 구현해 연출이 어긋난 것을 지적. 이후 시안의 마크업·스타일·스크립트를 **소스 수준으로 전수 분석**해 팔레트·치수·컴포넌트 구조를 값 단위로 대조하는 방식으로 전환했고, 이 원칙을 이후 모든 시안 이식에 적용.

정보 밀도 재설계 요구

"너무 많은 정보를 제공해서 한눈에 보기 어려운 디자인이라고 생각해. 포커류 베팅 게임이 기본적으로 갖고 있어야 할 요소들만 남겨서 쉽고 간단하게" — 정보 과다를 근거로 직전 디자인을 폐기하고, 카드·팟·베팅·액션만 남긴 미니멀 단일 컬럼으로 베팅류 3종을 통합(4절). 코드도 함께 감량(순감 약 1,100줄).

이식 범위에 대한 판단 요구

"모노크롬 레이즈도 같은 톤으로 손보면 어떻게 될지 궁금하네. 베팅류 게임이긴 하지만 엄연히 다른 전략 게임이니까 그 게임만의 느낌이 유지되어야 할 텐데. 프로토타입을 보여줄 수 있어?" — 본 코드를 고치기 전에 **동작하는 프로토타입**을 먼저 만들어 합의하는 절차를 요구. 공용 톤(무드·레이아웃)과 게임 고유 요소(흑백 타일·배치 트랙·스태시 경쟁)의 경계를 프로토타입에서 확정한 뒤 이식.

차별화 방향 결정 (사람의 감독)

"굳이 킥 포인트가 변형된 룰이 아니어도 될 것 같다는 생각이 들어. 디자인이 킥이 될 수도 있겠지. 십이장기도 일본의 동물장기를 장기화한 거니까, 우리도 디자인으로 승화시켜도 괜찮아 보인다." — AI가 제안한 룰 변형 15안 중 14안을 반려하고 방향을 전환한 결정. 밀림장기를 '백두 밀림'으로 재해석(짐승 기물 + 이동 방향 펄, 로직 변경 0줄)하고, 룰 킥은 블라인드 포커의 **블랙 핸드** 1건만 채택해 재학습까지 수행.

품질 기준 제시 (프로젝트 지침)

"추측하지 말 것 · 명세와 다르면 멈추고 물을 것 · 요청 범위 밖의 코드를 고치지 말 것 · 작업을 검증 가능한 목표 바뀔 때까지 반복할 것" — 저장소 CLAUDE.md에 상시 지침으로 두어 모든 작업에 적용.

3. 게임 AI 설계 (제품 속 AI)

공정성 원칙 — 모든 AI는 사람과 동일한 공개 정보만 사용합니다. 은닉 정보(상대 손패, 숨은 벽, 주사위 눈)는 코드 수준에서 접근하지 않으며, 각 ai.ts 파일 서두에 사용 가능한 정보의 경계를 명문화했습니다.

3.1 게임별 AI 기법

게임	핵심 기법
밀림장기	반복 심화 알파베타 탐색 + 치환표 (시간 예산 900ms, 통상 깊이 9~13)
테트라	공개 정보 기반 가상 플레이어 손패 추적 + 교환 기대값 평가, 0 카드 주입 방해 전략
블라인드 포커	CFR+ 자가대국 학습(전수 순회, 내시 균형 근사) + 카드 카운팅·베이지안 역추론 풀백
블라인드 홀덤	결과 표집 MCCFR 자가학습(공유 카드 위험 판독) + 내 이마 사후분포로 승률·페널티 확률을 정확히 적분하는 기대값 풀백
윷 대전	행렬게임 혼합전략(동시 선택 심리전) + 사람 선택 분포 학습 + 이동 가치 평가
윷과 거짓말	베이지안 거짓말 확률 추정 + 사람의 거짓 빈도·의심률 학습
리플렉트	반복 심화 알파베타 + 치환표, 레이저-왕 근접도 평가로 전진 동기 부여
모노크롬	외부 표집 MCCFR 자가학습(초·중반) + 베이지안 손패 추적·상대 신념 모델링·중반 완전 탐색 하이브리드
모노크롬 II	결과 표집 MCCFR 자가학습(균형 입찰) + 잔여 포인트 구간 추적(블로토 게임 이론) 풀백
야누스 포커	결과 표집 MCCFR 자가학습(면 선언+베팅) + 카드 카운팅·홀짝 제약·베이지안 뒷면 추정 풀백
암전 미궁	공개된 충돌/통과 정보 완전 기억 + 미지 간선의 확률 기대비용 최단경로 탐색
순환선	순환 완성 가능성 판정(증인 사이클) + 메모이제이션 승패 탐색(벽시계 예산 2.5초)
모노크롬 레이즈	결과 표집 MCCFR 2층 자가학습(배치 혼합 + 콜/폴드) + 칩 신호 기반 타일 추론 풀백
암수전(暗數戰)	은폐 기물 정체 확률 분포 + "움직인 기물은 지뢰가 아니다" 이동 이력 추론 + 위협 분석
히든 포물러	규칙 은행 208종 에 대한 후보 소거 귀납 추론 + 판별력을 최대화하는 능동적 수 제시 — 확정 시에만 버저

3.2 왜 게임마다 다른 기법인가 — 선택의 근거

동일한 기법을 15종에 일괄 적용하지 않았습니다. 게임의 수학적 구조가 최적 전략의 성격을 결정하기 때문입니다.

게임 구조	해당 게임	선택한 기법과 이유
-------	-------	------------

불완전정보 + 베팅/블러핑	블라인드 포커, 블라인드 홀덤, 모노크롬 I-II, 모노크롬 레이즈, 야누스 포커 (6종)	CFR 계열 자가학습. 최적해가 단일 수가 아니라 혼합 전략 이므로, 상대에게 착취당하지 않는 균형점을 자가대국으로 근사하는 것이 정석
완전정보	밀림장기, 리플렉트, 순환선	알파베타/완전 탐색. 숨은 정보가 없어 최적해가 확정적 — 혼합 전략이 필요 없고 깊이 탐색이 곧 실력
동시 선택(소규모)	윳 대전	행렬게임 내시 균형 해석적 계산. 매 턴 국소 게임이 작아 근사 학습보다 정확한 계산이 우월
상태 공간 과대	테트라, 암수전, 암전 미궁, 윳과 거짓말	베이지안 추론 + 도메인 휴리스틱. 정보집합이 조화 테이블 규모를 넘어 tabular CFR이 불가(스트라테고 계열은 신경망 기반 연구가 필요한 영역)
귀납 추론 퍼즐	히든 포물러	후보 소거 + 능동 실험. 전략 게임이 아니라 가설 검정 문제이므로 균형 학습의 대상이 아님

3.3 자가대국 강화학습 6종 — CFR 계열

포커 AI(Libratus/Pluribus)가 인간 최정상을 이긴 기법인 **CFR(Counterfactual Regret Minimization)** 계열로, 은닉 정보 게임 6종의 전략을 자가대국 학습했습니다. 신경망 없이 **표(tabular) 방식**이라 결과물이 곧 조회 가능한 전략표이고, **GPU 없이 일반 CPU에서 10분 내외로 재현**됩니다.

게임	표집 방식	반복 / 시간	정보집합	vs 기존 최강 AI
블라인드 포커	전수 순회 CFR+ (블랙 핸드 트리 병행)	6천 / 12.9분	14,566	70.0% (4,000판)
모노크롬	외부 표집 MCCFR	6천 / 31분	36,812	62.9% (2,000판)
모노크롬 II	결과 표집 MCCFR	2천만 / 9분	89,710	80.8% (2,000판)
모노크롬 레이즈	결과 표집 MCCFR (2층)	4천만 / 7.5분	55,614	67.3% (2,000판)
야누스 포커	결과 표집 MCCFR	3천만 / 3.3분	31,839	75.0% (500판·파산까지)
블라인드 홀덤	결과 표집 MCCFR (게임 단위 순회)	60만 게임·726만 핸드 / 22.9분	25,676	63.4% (2,000판)

모든 평가는 **실제 게임 엔진**으로 좌석을 교대하며 전체 게임을 치른 결과입니다. 전략표는 해당 게임 진입 시에만 지연 로드되는 별도 청크이며, 미로드·미적중 시 기존 휴리스틱으로 폴백해 동작 저하가 없습니다.

표집 방식을 게임마다 달리한 이유

- **전수 순회**(블라인드 포커) — 딜 380가지 × 작은 베팅 트리라 매 반복 전체를 정확히 순회할 수 있어 가장 정밀합니다. 변형 킥 **블랙 핸드**(텍당 2회, 아무도 카드를 못 봄)는 같은 딜 집합 위에서 **별도 정보집합 트리(키 점두 B)**로 함께 학습합니다. 첫 학습에서 균형이 "**선공 올인 97% · 콜 99%**"로 수렴해 — 무정보 대칭 상황은 쇼다운 기대값이 0이라 공격이 공짜가 되는 구조 — 블랙 핸드가 스택 전체를 건 동전 던지기가 됨을 발견했고, **레이즈 상한(1인 4칩)**으로 룰을 보정한 뒤 **재학습**했습니다. 학습된 균형이 게임 디자인의 결함을 역으로 찾아낸 사례입니다.

- **외부 표집**(모노크롬) — 손패가 9→1장으로 줄어드는 구조라 순회자 전 분기가 유한하게 끝납니다. 은닉 대상이 카드가 아니라 **행동 자체**인 게임이라 정보집합을 공개정보(내 손패 비트마스크 × 상대 잔여 흑/백 장수 × 점수차 × 선/후)로 압축했습니다.
- **결과 표집**(나머지 4종) — 모노크롬 II는 최대 18라운드 × 후보 12개로 전 분기가 지수 폭발하므로, 반복마다 궤적 하나만 걷고 중요도 가중으로 리그렛을 추정합니다. 덕분에 2천만 회를 9분에 완주했습니다.
- **결과 표집 + 게임 단위 순회**(블라인드 홀덤) — 다른 게임은 매 반복 새 딜에서 한 핸드만 뽑지만, 이 게임은 **잔량 카운팅·스택 변화·이월 패**가 **정보집합에 영향을 줍니다**. 한 핸드만 뽑으면 "덱이 막 시작된 상태"만 배우게 되므로, 한 게임을 파산까지 굴리며 핸드마다 갱신해 그 상태들이 분포에 자연스럽게 들어오게 했습니다.

확률을 버킷으로 쓴 이유 — 학습과 실전의 정보량이 달라도 되게

블라인드 홀덤의 정보집합 키는 원시 카드가 아니라 **계산된 확률**을 버킷으로 씁니다(내 승률 8단계, 폴드 페널티 확률 4단계, 상대 족보 4단계, 공유 카드 위험 프로필 3단계). 공유 카드가 공개이므로 상대 족보는 확정이고, 모르는 것은 내 이마 하나뿐이어서 **승률을 사후분포로 정확히 적분**할 수 있기 때문입니다.

이 선택에는 부수 효과가 있습니다. 학습 중에는 덱 초반 분포로 계산된 승률이, 실전에서는 카운팅이 누적된 더 정확한 승률이 됩니다. 원시 카드를 키로 썼다면 이 차이가 곧 모델 불일치가 되지만, **"내 승률이 대략 x다"**라는 버킷의 의미는 **어떻게 계산했든 동일**하므로 학습한 전략이 그대로 유효합니다. 실측 정보집합 적중률 **99.0%**가 이를 뒷받침합니다.

설계 원칙 — 모델 불일치 차단

학습기가 게임을 따로 흉내 내면 학습한 전략이 실전에서 빗나갑니다. 그래서 트레이너가 **실제 엔진의 함수를 그대로 호출**해 시뮬레이션하고, 정보집합 키를 계산하는 코드도 게임 SI와 **같은 모듈을 공유**하도록 했습니다. 그 결과 실전 정보집합 적중률이 91~99.8%로 측정됩니다 — 학습한 상황이 실전에 그대로 나온다는 뜻입니다.

실패에서 배운 것 — 모노크롬 레이즈 2층 학습

이 게임은 비공개 배치(순서 10! × 칩 분배)와 콜/폴드 결정의 2층 구조입니다. 배치 공간은 전수 학습이 불가능해 **전략적으로 구분되는 배치 템플릿 48종**(정석/톱 스파이크/블러프/함정 등)을 정의하고 그 위의 혼합 전략을 학습했습니다.

1차 학습 결과는 **승률 43.2%로 실패**였습니다. 원인 분석 결과 결정 적중률이 49.5%에 불과했는데, 템플릿끼리만 자가대국을 시켜 **상대의 임의 배치가 만드는 상황을 학습한 적이 없었기** 때문입니다. ① 상태 버킷을 굵혀 일반화하고 ② "상대 베팅이 자기 잔여 중 몇 번째로 큰가"라는 핵심 추론 신호를 키에 추가하고 ③ 학습 상대의 25%를 임의·휴리스틱 배치로 섞어 재학습한 결과, 적중률 99.8% / **승률 67.3%**로 개선됐습니다.

수치를 해석하는 법 — 62.9%는 큰 격차인가

승률만으로는 크기를 알 수 없어 **기준선을 측정**했습니다. 모노크롬에서 기존 최강(hard) AI가 한 단계 아래(normal) AI를 이기는 비율은 **65.3%**였습니다. 즉 이 게임에서 "난이도 한 등급"의 격차가 65% 수준이고, 학습 AI는 그 최강 AI 위에 다시 62.9%를 기록한 것이므로 **등급 하나를 더 쌓은 셈**입니다. 서로 같은 패로 시작하는 은닉 정보 게임은 구조적으로 승률이 50%에 수렴하며, 포커 AI 연구도 압도적 승률이 아니라 대량 표본의 기대값 우위로 실력을 입증합니다. 2,000판에서 62.9%는 95% 신뢰구간 ±2.1%로 우연일 확률이 사실상 0입니다.

부수적으로 관찰된 현상도 있습니다. 모노크롬 학습 전략은 **약한 상대(normal)**에게는 57.8%로, 기존 hard(65.3%)보다 오히려 낮았습니다. CFR이 배우는 것은 "누구에게도 지지 않는" 균형 전략이지 "약자를 최대로 착취하는" 전략이 아니라는 이론적 성질이 실측으로 확인된 것입니다. 그래서 모노크롬은 **초·중반은 균형 전략, 중반은 기존의 정밀 벨리프 완전 탐색**이라는 하이브리드로 탑재했습니다 — 순수 정책만 쓰면 48.8%였으나 하이브리드는 62.9%였습니다.

재현: `npm run cfr:train / cfr:train:mono / cfr:train:m2 / cfr:train:raise / cfr:train:janus / cfr:train:holdem` (각 `cfr:eval:*` 로 평가). 학습·평가 스크립트 전부 `scripts/cfr/` 에 포함.

3.4 대표 사례 심화

모노크롬 — 상대 신념 모델링. 상대의 남은 타일 분포를 색·승패 기록으로 베이지안 갱신하는 데 그치지 않고, "상대가 내 공개 정보로 무엇을 알고 있는가"까지 모델링해 블러핑 여지를 계산합니다. 중반 완전 탐색에는 사용자의 라운드별 제시 성향(localStorage 누적)이 가중치로 반영됩니다.

히든 포물러 — 오답 없는 귀납 추론과 규칙 은행 208종. AI는 숨은 규칙을 읽지 않습니다. 규칙 은행의 후보들을 힌트와 대조해 소거하고, 후보가 하나로 확정되거나 모든 후보의 답이 일치할 때만 버저를 누릅니다. 수 제시는 후보 판별력을 최대화하는 능동 실험으로 선택합니다 — 사람과 같은 방식으로 추론하되 실수만 없는 상대입니다.

한 판에 규칙 11개를 쓰므로 초기 24종으로는 두세 판이면 전부 노출됐습니다. 그래서 **12개 계열을 매개변수로 전개해 208종**으로 확충했습니다(사칙·나머지·자릿수·이어쓰기·수론·비트연산·숫자 모양·조건부·수열 등, 원작 규칙은 보존). 대량 추가는 필연적으로 결함을 낳으므로 **자동 감사 도구**(`npm run rules:audit`)를 함께 만들어 ① 불량 출력 ② 행동 서명 기반 완전 중복 ③ 16힌트 내 판별 가능성을 검사했습니다. 감사는 실제로 결함 9건을 잡아냈습니다 — 특히 "두 수의 끝자리를 서로 바꿔 더하기"는 **맞바꿔도 합이 불변**이라 '합'과 동일한 규칙이었습니다. 수정 후 최종: 208종, 중복 0, 정답 확정 평균 2.10힌트, 16힌트 판별 실패 0.1%. AI는 후보 소거식이라 규칙 추가에 **코드 수정 없이 자동 대응**합니다.

윳과 거짓말 — 성향 학습. 의심으로 공개된 선언만을 근거로 사람의 거짓말 빈도와 값별 분포, 값별 의심률을 학습해 베이즈 추정에 반영합니다. 믿어준 선언의 실제 값은 AI도 영원히 알 수 없습니다(공정성 원칙).

3.5 표기의 정확성 — AI 설명 문구 감사

CFR 전략을 탑재하면서 5개 게임의 의사결정 주체가 바뀌었고, 기존의 "당신의 패턴을 학습합니다"류 안내 문구가 **더 이상 사실이 아니게** 되었습니다(성향 학습이 폴백 경로에만 남음). 15개 게임의 셋업 문구를 실제로직과 전수 대조해, 사실과 어긋난 4건을 자가학습 균형 전략 안내로 교체하고 모노크롬은 하이브리드 구조를 정확히 표기했습니다. 나머지 10종은 문구가 로직과 일치함을 확인했습니다. **구현되지 않은 지능을 광고하지 않는 것도** AI 활용의 일부라고 판단했습니다.

4. 시스템 아키텍처

- **스택** — TypeScript + React 19 + Vite. 게임 엔진·AI는 외부 라이브러리 없이 순수 TypeScript.
- **구조** — 게임마다 `engine.ts` (순수 함수 룰) / `ai.ts` (대전 AI) / `view.ts` (좌석별 정보 마스크) / 화면 컴포넌트로 분리.
- **온라인 멀티** — 서버 없이 공개 nostr 릴레이를 데이터 통로로 사용(방 코드에서 유도한 키로 AES-GCM 암호화). 호스트 권위 방식 + 좌석별 관점 뷰만 전송. 재연결·메시지 순번 순서 보장·재전송 유실 복구·좌석 고정을 전송 계층에서 처리.

- **공개 방 목록·채팅** — 같은 릴레이를 게시판처럼 사용해 대기 중인 방을 게임별로 공고하고, 목록에서 클릭 한 번으로 입장합니다(공고가 끊기면 자동 제거). 채팅은 게임 메시지와 같은 암호화·순서 보장 파이프라인에 사이드채널로 실어, **14종 게임 프로토콜을 수정하지 않고** 전 게임에 장착했습니다.
- **학습 전략 자산** — CFR 전략표는 게임별 JSON으로 빌드되어 **해당 게임 진입 시에만 지연 로드**되는 별도 청크로 분리됩니다(gzip 129KB~941KB). 메인 번들 크기에 영향이 없고, 로드 실패 시 휴리스틱으로 폴백합니다.
- **치팅 방지** — 동시 선택·비밀 설계는 **커밋-리빌**(SHA-256 해시 커밋 후 공개·검증)로, 주사위는 **분산 생성**(굴리는 쪽 난수 커밋 + 상대 난수 합산)으로 호스트조차 엿볼 수 없게 설계.
- **테마 시스템** — CSS 변수 토큰 기반 5종 테마. 게임 의미 색(흑/백 타일, 4색 카드, 레이저 등)은 테마와 무관하게 불변으로 두어 룰 가독성을 보존.
- **베팅류 공용 UI 계층** — 포커 계열 4종(블라인드 포커·블라인드 홀덤·야누스 포커·모노크롬 레이즈)은 공용 모듈(`shared/pokerui.tsx`) 하나에서 레이아웃·좌석·팟·슬라이더·무드를 공급받습니다. 화면은 카드·팟·베팅·액션만 남긴 단일 컬럼($\leq 470\text{px}$)이고, 하단 액션 패널은 **지금 가능한 조작만** 렌더합니다. 각 게임의 고유 요소는 게임 쪽에 둡니다 — 야누스의 면 선언·3D 플립 카드, 모노크롬 레이즈의 흑백 타일·10칸 배치 트랙·설계 화면. 화면 높이에 따라 카드·숫자 치수를 한 단계 줄이는 압축 규칙으로 **스크롤 없이 한 화면에** 들어옵니다.
- **무드 시스템** — 베팅류 4종은 테이블 분위기(느와르·펠트·아이보리)를 게임 간 공유하며 선택은 저장됩니다. 무드는 배경·강조색만 바꾸고, **모노크롬 레이즈의 흑백 타일은 무드와 무관하게 무채색**으로 고정해 게임의 정체성과 룰 가독성을 보존합니다.

5. 외부 에셋 / 오픈소스 출처

구분	이름	라이선스 / 비고
라이브러리	React 19, React DOM	MIT
라이브러리	Vite, @vitejs/plugin-react, TypeScript	MIT / Apache-2.0 (빌드 도구)
라이브러리	nostr-tools	MIT — 온라인 대전 이벤트 서명/검증
인프라	공개 nostr 릴레이 (relay.damus.io, nos.lol, relay.primal.net)	공개 릴레이 — 게임 데이터는 암호화되어 경유만 함
폰트	Pretendard Variable	SIL OFL 1.1
폰트	Space Grotesk · Chakra Petch · Orbitron · Cinzel (Google Fonts)	SIL OFL 1.1 — 테마별 디스플레이 폰트
그래픽	이모지(게임 아이콘 등)	플랫폼 기본 이모지 — 별도 이미지 에셋 없음
그래픽·사운드	기타 이미지/사운드 에셋	사용하지 않음 — 모든 비주얼은 CSS/SVG로 직접 제작

룰 출처 표기

각 게임은 더 지니어스 시리즈 방영 게임의 룰(나무위키 '더 지니어스/역대 게임' 및 개별 게임 문서에서 확인)과 그 원류 보드게임(동물장기, Khet, 베니스 커넥션, 마법의 미로, 찰오찰오, 스트라테고, 파라오코드 등)을 참고해 **룰을 재해석·재구현**한 것입니다. 원작의 그래픽·명칭·소스는 일절 사용하지 않았으며, 게임 이름도 모두 새로 지었습니다. 변형한 규칙은 게임 내 설명서와 `docs/GAME_RULES.md`에 원작과의 차이로 명시되어 있습니다.